॥वसुधैव कुटुम्बकम्॥

SYMBIOSIS

INSTITUTE OF TECHNOLOGY, PUNE

Digital Lock With Multi-Factor Authentication

Submitted by:-

Sr No.	Name	PRN
1.	Arnav Kumar	25070123503
2.	Mahi Shendre	25070123504
3.	Aditya Varudkar	25070123506
4.	Chetan Gawale	25070123507

Department of Electronics and Telecommunication Engineering

AY 2025-26 Semester - III Batch 2024 - 2028

Acknowledgement

We would like to express our sincere gratitude to **Dr. Pritesh Shah**, Head of the Department of Electronics and Telecommunication at Symbiosis Institute of Technology (SIT), Pune, for his invaluable support and for fostering an environment that encourages practical learning and project development.

We are also deeply grateful to our project guide, **Dr. Sumit Kumar**, for his expert guidance, constant encouragement, and insightful feedback throughout the duration of this project. His mentorship was instrumental in the successful completion of our Project.

Finally, we thank the faculty and staff of the ENTC department for providing the necessary resources and support.

Abstract

With the rapid rise of automation and smart devices, ensuring reliable and secure access control has become an essential aspect of modern embedded systems. Traditional single-layer authentication methods such as passwords or RFID tags alone are increasingly vulnerable to breaches, making multi-factor authentication a more robust and necessary alternative for safeguarding both digital and physical environments.

This project presents a Triple Authentication Digital Lock System built on the ESP32 microcontroller, integrating RFID-based identification, keypad PIN entry, and Telegram-based One-Time Password (OTP) verification. The system validates users sequentially through these three stages, granting access only upon successful completion of all checks. Implemented in MicroPython, the prototype employs an MFRC522 RFID module, 4×4 matrix keypad, and Wi-Fi connectivity for Telegram integration. It also includes real-time alert notifications, LED status indication, and an automatic timeout mechanism for enhanced reliability. The project demonstrates how low-cost microcontrollers can effectively combine embedded hardware and IoT capabilities to deliver secure, scalable, and smart access solutions for real-world applications.

Table of Contents

Sr. No.	Description	Page No.
1	Introduction	1
2	Objective	2
3	Circuit Design	3
4	Implementation	4
5	Testing & Results	7
6	Conclusion and Learning	10

1. Introduction

In today's world of rapidly advancing technology, the need for secure and intelligent access control systems has become more important than ever. From residential buildings and office environments to laboratories and data centers, preventing unauthorized entry has emerged as a critical concern. Traditional single-layer authentication mechanisms such as physical keys, RFID cards, or numeric passwords are increasingly vulnerable to duplication, hacking, and human error. These limitations have led to the evolution of multi-factor authentication (MFA) , a process that requires users to verify their identity through multiple independent methods before access is granted.

Multi-factor authentication enhances system security by combining multiple credentials — typically something the user *has* (e.g., RFID card), something the user *knows* (e.g., password or PIN), and something the user *receives* (e.g., a time-sensitive OTP). This layered verification approach ensures that even if one factor is compromised, unauthorized access remains difficult. The principle of MFA has become the foundation of secure systems used by banks, research labs, and IoT-based smart environments.

In this project, a Triple Authentication Digital Lock System has been designed and implemented using the ESP32 microcontroller as the core processing unit. The system integrates three different security layers. RFID card verification, keypad-based PIN entry, and Telegram-based One-Time Password (OTP) verification to ensure a robust and reliable security mechanism. Each authentication stage is dependent on the successful completion of the previous one, thereby reducing the chances of intrusion.

The **ESP32** was selected due to its dual-core processing capability, built-in Wi-Fi and Bluetooth modules, and ease of programming with **MicroPython**. These features make it ideal for developing IoT-enabled embedded systems where both local hardware interfacing and cloud-based communication are required. The **MFRC522 RFID module** is used for card-based identification via SPI communication, while a **4×4 matrix keypad** provides user input for PIN and OTP entry. Visual indicators in the form of red and green LEDs display system states such as locked, unlocked, or invalid access.

A unique feature of this project is the integration of **Telegram messaging** as part of the authentication process. Upon successful PIN entry, the ESP32 generates a six-digit random OTP and sends it to the user via a Telegram bot using the Telegram API over Wi-Fi. The user must then enter the received OTP on the keypad to complete authentication. Telegram also provides real-time notifications for system events, such as invalid card detection, failed login attempts, or successful unlock confirmations, allowing remote monitoring and control.

The entire system operates sequentially: the user first scans an RFID card, then enters a 4-digit PIN, and finally inputs the OTP received on Telegram. Any mismatch at any stage prevents access and triggers a visual alert along with a message notification. The implementation demonstrates how embedded systems, when integrated with IoT services, can deliver advanced, multi-layered, and user-friendly security solutions.

Overall, this project showcases how **low-cost hardware and open-source software** can be combined to build a highly effective smart locking system. It not only meets the requirements of a modern access control mechanism but also provides a scalable foundation for future enhancements, such as fingerprint sensors, mobile app control, and cloud-based data logging.

2. Objectives

The primary objective of this project is to design and develop a **multi-factor authentication digital lock system** that enhances the security and reliability of access control using the ESP32 microcontroller. The system combines multiple layers of verification to minimize the risk of unauthorized access and to demonstrate the integration of embedded hardware with IoT-based communication.

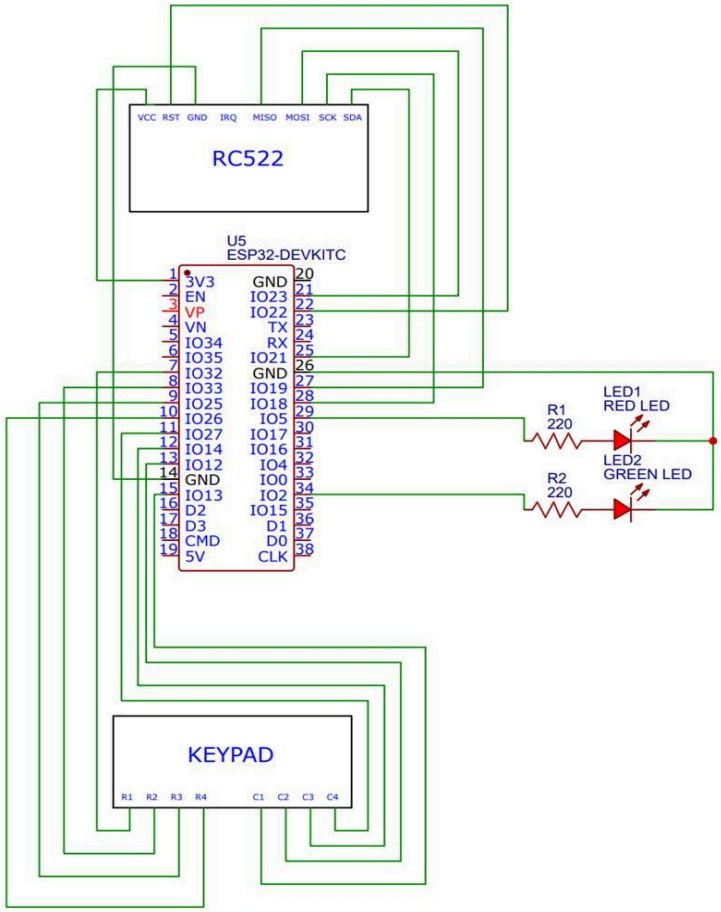
The specific objectives of the project are as follows:

1. **To design a triple authentication system** that uses three independent verification methods — RFID card detection, PIN entry through a keypad, and OTP verification via Telegram — to ensure a high level of access security.
2. **To interface the ESP32 microcontroller with external hardware modules** such as the MFRC522 RFID reader, a 4×4 matrix keypad, and LED indicators for visual feedback, demonstrating proper use of GPIO and SPI communication protocols.
3. **To integrate IoT functionality using Wi-Fi connectivity**, enabling the ESP32 to communicate with the Telegram cloud API for sending One-Time Passwords (OTP) and system notifications in real-time.
4. **To implement a user authentication workflow** where each stage (RFID → PIN → OTP) must be successfully completed before unlocking, ensuring logical sequencing and validation of inputs.
5. **To provide visual status feedback** through red and green LEDs, representing the lock's state (locked, unlocked, or access denied), and an internal LED for Wi-Fi connectivity indication.
6. **To ensure system robustness** by including features such as timeout handling, input cancellation (via keypad), and invalid card detection alerts through Telegram messages.
7. **To demonstrate the practical application of MicroPython** in embedded system design, emphasizing modular coding, real-time operation, and cloud communication.
8. **To develop a scalable and low-cost security prototype** that can be adapted for various environments, including residential, institutional, and industrial access control systems.

3. Circuit Design

The circuit design of the ESP32-based Digital Lock System integrates multiple hardware modules to achieve a secure, multi-factor authentication process. The ESP32 serves as the main processing unit that coordinates inputs from both the RFID reader and the 4×4 matrix keypad, performs logical verification, and controls the visual indicators. The RC522 RFID module allows contactless identification of authorized users, while the keypad provides secondary PIN or OTP verification. In addition to these input modules, the system is equipped with two indicator LEDs and an onboard Wi-Fi interface that enables communication with a Telegram bot for remote verification and alert notifications.

The overall design emphasizes modularity, compactness, and reliability. Each peripheral component communicates with the ESP32 through dedicated GPIO pins, ensuring minimal interference between devices. The SPI protocol is used for communication between the ESP32 and the RC522 module, offering fast and stable data transfer. The keypad is interfaced through general-purpose I/O pins, configured for row-column scanning. The red and green LEDs are connected through current-limiting resistors to visually indicate system states such as “locked”, “unlocked”, and “authentication failed.



4. Implementation

4.1. Hardware Assembly

1. ESP32 Placement
   The ESP32 development board was placed on a breadboard with sufficient space on both sides for jumper connections. The USB cable connected the board to the computer, which also supplied 5V power.
2. RC522 RFID Module Connection
   The RC522 module was interfaced with the ESP32 via SPI communication.

RC522 PINs	ESP32 PINs
3.3V	3V3
GND	GND
SCK	GPIO18
MOSI	GPIO23
MISO	GPIO19
SDA	GPIO21
RST	GPIO22

3. Keypad Connection

Keypad PINs	ESP32 PINs
R1	GPIO 32
R2	GPIO 33
R3	GPIO 25
R4	GPIO 26
C1	GPIO 27
C2	GPIO 14
C3	GPIO 12
C4	GPIO 13

4. LED Indicators
   Two LEDs were added to indicate system states:
   - Red LED (GPIO 5) → Indicates *Locked / Denied access*.
   - Green LED (GPIO 2) → Indicates *Unlocked / Access granted*.
     Each LED was connected in series with a 220Ω resistor to ground.
5. Power Supply
   The ESP32 and connected modules were powered directly through the USB port from the computer. The RC522 module was powered via the 3.3V pin, as it is not 5V-tolerant.

4.2. Software Setup

1. Installing MicroPython and Thonny IDE
   The ESP32 was flashed with the latest MicroPython firmware. Thonny IDE was used for writing, editing, and uploading scripts. The interpreter was set to "MicroPython (ESP32)" and the correct COM port was selected.
2. Uploading Required Libraries
   The following Python files were uploaded to the ESP32 using Thonny's "Save to device" feature:
   - main.py — Main control logic and state machine
   - mfrc522.py — RFID driver for MicroPython
   - keypad.py — Keypad scanning driver
   - config.py — Stores Wi-Fi, Telegram, and authentication parameter
3. MicroPython Firmware Verification
   A simple LED blink program was first uploaded to confirm the ESP32 was functioning correctly before proceeding with the final integration.

4.3. Telegram bot setup

1. Create a bot
   - In Telegram app, message @BotFather → /newbot → name and username → receive Bot Token. Save the token.
2. Get your Chat ID
   - Start a chat with the created bot and send a message.
   - Use browser:
   - https://api.telegram.org/bot<YOUR_TOKEN>/getUpdates

Find "chat":{"id":<YOUR_CHAT_ID>} in response.

3. Store credentials
   - Put BOT_TOKEN and CHAT_ID in config.py. Protect these keys; do not publish them publicly.

4.4. Code structure & important functions

1. config.py
    - Central place for constants:
    - WIFI_SSID = "Arnav's Android"
    - WIFI_PASS = "cmfphone1"
    - BOT_TOKEN = "8412858567:AAGOhM89r01IXHAtolz3Bg6FsiloI29ledM"
    - CHAT_ID = "1327556061"
    - VALID_UIDS = ["11 B5 F9 03", "71 DC B9 89"]
    - PIN_CODE = "0000"
    - LED_PINS = {"red":5, "green":2, "wifi":2}
2. mfrc522.py
    - Use a MicroPython-compatible MFRC522 driver (your working version). Ensure its SPI init uses the pins you wired.
3. keypad.py
    - Matrix scanning driver: configure rows as outputs (drive low/high) and columns as inputs with pull-ups or vice versa. Provide get_key() to return latest key or None.
4. main.py - high-level logic:
    - Connect to Wi-Fi and sync time if needed.
    - Initialize RFID, keypad, LEDs.
    - State machine:
        - LOCKED: wait for RFID; if UID in VALID_UIDS → WAITING_FOR_PIN.
        - WAITING_FOR_PIN: collect keypad digits; # = submit, * = clear; if PIN correct → generate OTP and send to Telegram; WAITING_FOR_OTP.
        - WAITING_FOR_OTP: collect OTP input; on correct → UNLOCKED (green LED ON); incorrect → reset to LOCKED.
        - UNLOCKED: auto relock after timeout.
    - Telegram functions:
        - send_telegram_message(text) - simple urequests.get(url) call.
        - check_telegram_commands() - poll /getUpdates for authorized commands. Use offset to avoid reprocessing.
5. Error handling & robustness
    - Add timeouts for waiting states.
    - Debounce keypad inputs.
    - Catch exceptions on network requests and retry.
    - Persist secret/UID list in a file if you need runtime modifications.

5. Testing & Results

5.1. Testing procedure

1. Initial functional tests
    - Scan authorized card → confirm REPL prints UID and transitions to enter PIN.
    - Enter wrong PIN → confirm it resets to locked and sends Telegram alert (if enabled).
    - Enter correct PIN → confirm OTP is sent to Telegram.
    - Enter correct OTP → confirm green LED ON and UNLOCKED state.
    - Enter wrong OTP → system resets and alert sent.
2. Edge-case tests
    - Scan unauthorized card → immediate rejection and Telegram alert.
    - Partial keypad input then * → confirm it clears input.
    - Repeated failed attempts → confirm timeout / lockout behavior if implemented.
3. Network tests
    - Turn off Wi-Fi (or change SSID) → confirm Wi-Fi LED blinks fast and Telegram functions fail gracefully.
    - Recover Wi-Fi → confirm reconnect behavior and LED slow blink.
4. Power tests
    - Unplug and re-plug → confirm device boots cleanly and main.py auto-starts.

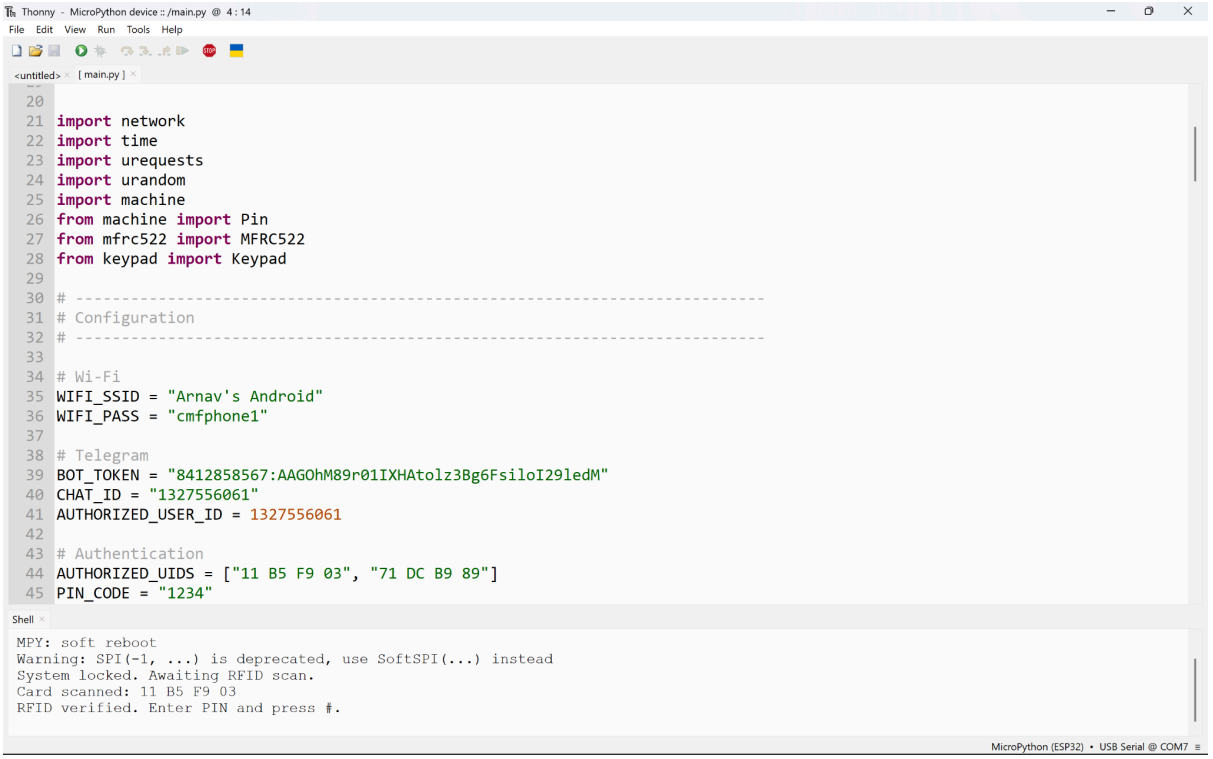
5.2. Troubleshooting common issues

1. RC522 not detected
    - Check SPI wiring (MISO/MOSI/SCK/CS/RST), ensure 3.3 V supply.
    - Use a short SPI wire length and common ground.
2. Keypad not reading
    - Confirm row/column pin directions and pull-ups.
    - Add small delays in scan loop for debouncing.
3. Telegram network failures
    - Ensure correct BOT_TOKEN and CHAT_ID.
    - Use urequests exceptions to log error and retry after a short delay.
4. Serial monitor disconnects when opening
    - Some USB-serial adapters change port on open; reselect the COM port in Thonny.
    - Avoid opening multiple serial monitors simultaneously.

5.3. Results



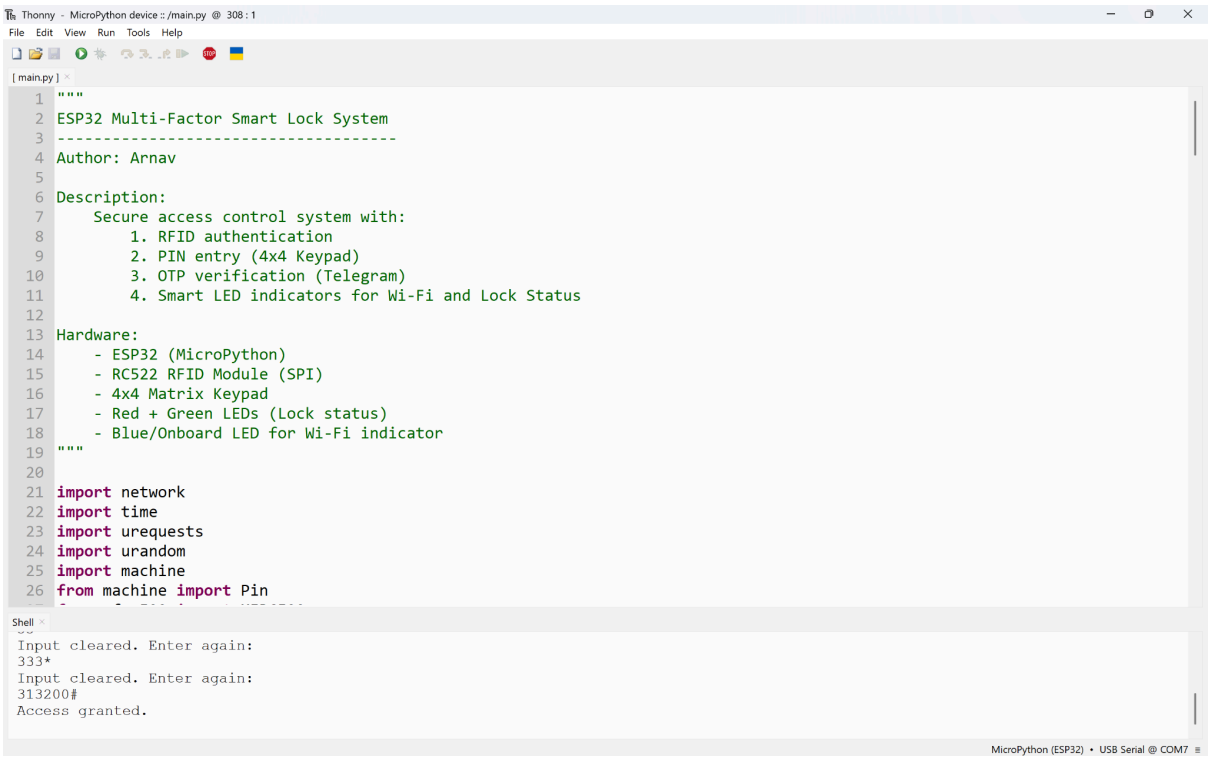
Locked State



RFID Card Verified



Pin Verified



OTP Verified & Access Granted

6. Conclusion and Learning

6.1. Conclusion

The multi-factor authentication digital lock system using ESP32, RC522 RFID module, 4×4 matrix keypad, and Telegram integration was successfully designed, implemented, and tested.
The project demonstrated the practical application of both hardware interfacing and secure authentication logic within an embedded IoT environment.

Through a combination of RFID-based identification, PIN verification, and Time-based One-Time Password (OTP) delivered via Telegram, the system ensures enhanced security compared to traditional single-factor locks. Each authentication layer contributes to a hierarchical verification process that minimizes unauthorized access.

The ESP32 microcontroller served as the central processing unit, handling communication between all modules, Wi-Fi connectivity, and cloud-based messaging. The RFID reader provided unique identification through card UIDs, while the keypad enabled manual PIN and OTP entry. The inclusion of Telegram not only improved usability but also introduced real-time monitoring, remote notifications, and command control capabilities.

Testing confirmed that all modules operated reliably under different conditions. Valid RFID cards were identified instantly, invalid cards were rejected with immediate Telegram alerts, and OTP verification remained stable and accurate. The LED indicators provided clear visual feedback on the system's state, while the internal LED reflected Wi-Fi connectivity in real time. Overall, the system achieved its goal of demonstrating a functional, IoT-based smart locking mechanism combining both hardware reliability and cloud connectivity.

6.2. Learnings

This project provided valuable exposure to the integration of hardware and software in embedded systems, particularly involving real-time communication over Wi-Fi. **Key technical learnings include:**

1. MicroPython programming on ESP32 - Understanding how to control peripherals (SPI, GPIO, and matrix keypad) using a lightweight scripting language suitable for rapid development.

2. RFID communication (SPI interface) - Learning how RFID tags transmit unique identifiers and how to implement authentication logic based on those UIDs.

3. Matrix keypad interfacing - Scanning multiple inputs using digital I/O pins and debouncing for accurate key detection.

4. IoT connectivity and Telegram API - Implementing HTTP-based API calls to send and receive messages, demonstrating real-world Internet of Things applications.

5. State-machine design - Structuring the system into logical states (LOCKED, WAITING_FOR_PIN, WAITING_FOR_OTP, UNLOCKED) for efficient control flow.

6. Error handling and timeout logic - Implementing safety measures to prevent misuse, unauthorized access, or system hanging during network failure.

7. Practical debugging and testing - Identifying and resolving issues related to serial communication, Wi-Fi reliability, and power management during continuous operation.

Beyond the technical aspects, this project also reinforced the importance of systematic design, documentation, and modular programming practices. The step-by-step implementation, testing, and iterative debugging closely mirrored a real engineering development process.

In conclusion, this mini-project not only strengthened the understanding of digital logic design concepts but also showcased how modern embedded controllers like the ESP32 can bridge electronics, networking, and cloud services into one unified smart system.